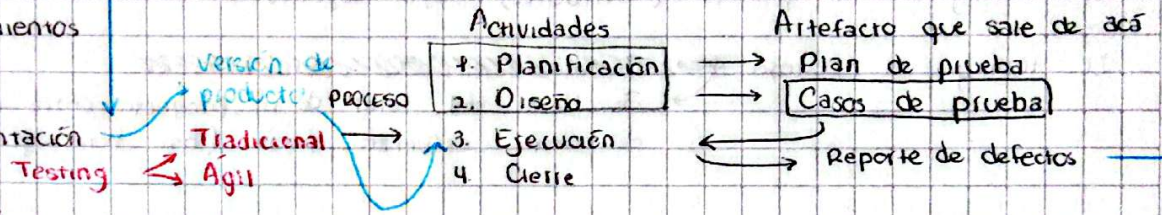


Testing / Prueba

Proceso [PUD]

- Requerimientos
- Análisis
- Diseño
- Implementación
- **Prueba**
- Despliegue



El cierre lo hace testing.

En la US defino Pruebas de Usuario, que son escenarios para el caso de prueba.

Reporte de defectos: lista con los errores encontrados, deben corregirse y mandarse de nuevo con una nueva versión de producto hasta que tenga un nivel de calidad.

Planificación y diseño es el 80% del esfuerzo del testing que sí puedo adelantar sin una nueva versión. Se acorta el tiempo. Necesito los requerimientos.

Los casos de pruebas deben estar escritos Sí o Sí. Excel, word, pizarra, herramienta, automatizada, etc. Jira, devOps, etc. Sin importar cómo.

los artefactos deben estar sí o sí, sin importar si uso tradicional o ágil.

Conceptos:

Testing metodológico: Usa casos de prueba prediseñados. Se planifica. Tiene proceso tiene casos de prueba.

Testing ad hoc: No se puede reproducir. Un error que no se puede reproducir no es un error. No metodológico. No sistemático. No tiene proceso. Me siento a probar sin casos de prueba.

Prueba: validar o verificar

Validar: Proceso de adecuación. Ver si hace lo que realmente tiene que hacer. Son más importantes. (error de req.) hay que rehacer muchas cosas.

Verificar: Ver si lo que hace lo hace bien. (error de programación)

Tengo que poder hacer ambas cosas. Puede pasar una y no la otra, ambas o ninguna.

Error de req. → muy caro

Error de programación → más económico de corregir.

Error vs defecto

A veces los errores y las fallas no generan mal funcionamiento en el sistema. Las fallas ocurren en el software como consecuencia de un error que o defecto.

Defecto: algo que se trasladó de una etapa anterior, donde se construyó.
Testing encuentra defectos porque no se originaron en la etapa de testing.

Algo es un error cuando no se trasladó.

Ejemplo programación de pares. Más barato en términos económicos. Pair programming.

El testing es un proceso (destrutivo) cuyo objetivo es la detección de defectos

El testing es costoso. ~~Se lleva del total de un proyecto entre el 20% y 50% del costo~~
Se lleva del total de un proyecto entre el 20% y 50% del ~~costo~~ esfuerzo total (hs. horas - persona)

Esfuerzo de un proyecto → 80% del costo total es esfuerzo.

Implementación 25%

Prueba 30% a 50%

Si nadie hace testing es el usuario final el que termina haciendo testing.

Edad del defecto: tiempo que transcurre desde que se detecta hasta que se corrigió.
a más edad tenga más costoso es.

Severidad del Defecto No tiene que ver con lo que a mí me cuesta corregirlo, tiene que ver con la gravedad que causa en el uso. la asigna testing.

Disqueante o invalidante → no se puede usar el sistema.

Grave.

Leve

Cosmética → Formateo de los campos, ortografía, puntos, comas, etc.

Mejora → NO es un defecto. Es algo que al tester le parece mejor.

se pueden asignar
tiempos máx
para resolver cada
tipo de severidad

Traking de defectos, Jira, devops,

Prioridad: La define el cliente. Si todo es prioridad alta no estoy priorizando nada.

Alta

Media

Baja

Niveles de testing:

Prueba de aceptación o de Usuario

Sistema

Integración o de interfaces → el resultado es un build o nueva versión de producto.

Prueba unitaria → se integra al TDD

primer nivel de testing

Siempre el desarrollador

Requerimientos

Análisis

una unidad de desarrollo no
debería probar su propio trabajo

- subjetividad.

Diseño

- actitud

Implementación UNITARIA

INTEGRACIÓN, cualquiera de los dos

Prueba SISTEMA

Despliegue ACEPTACIÓN

Ambientes de software → De mínima debe haber 3.

Desarrollo → Pruebas unitarias y de integración

Prueba → Limpio, separado, etc. Pruebas de sistema

Pre-producción → Pruebas de aceptación de usuario

Producción → si no existe pie lo hago en producción

La gente de desarrollo no tiene acceso a los ambientes de prueba.

Ciclo de prueba:

Ciclo cero: el primer ciclo de prueba que se ejecuta.

Ciclo 1-n

Ideal → 2 ciclos de prueba por cada versión que entra a testing.
el ciclo cero siempre existe, encuentro los errores, vuelvo a validar y está todo corregido.

Estrategia de prueba

- Ejecuto el ciclo de cero, rep. defectos, desarrollo, nueva versión.
Hago el ciclo 1 mirando el reporte.
Sin regresión → Más rápida / económica
- Vuelvo a hacer ciclo 1 desde cero, sin importar el reporte.
Con regresión → Pejor. Estadísticamente cada error que se soluciona implementa 3 más.

¿Qué se prueba? Los requerimientos funcionales y no funcionales. Pruebo ambos.

difíciles si no tengo la misma infraestructura de producción.

Tipos de pruebas

Describen el foco del producto que quiero probar.

Requerimientos → Funcionales → **Requerimientos funcionales de Operación Normal**
→ **Requerimientos funcionales de Operación Negativa**
→ **Pruebas de negocio**

~~Requerimientos~~ **Documentación**

No necesariamente doc, video, etc.

Manual de Usuario → Describe exactamente de la forma en la que funciona el usuario
Manual de Configuración

Requisitos → No funcionales → ~~Pruebas de performance~~

- Prueba de performance → tiempos de respuesta máx o mín. Depende de la cant. de usuarios a la que me comprometi.
- Prueba de interface → lo que primero ve el usuario.
- Prueba de escala compiera → todo lo que yo me comprometi, soft. hard. concurrencia.
ej. 500 usuarios concurrentes
- Prueba de stress → prueba con más ej. 1000 usuarios concurrentes.
- Prueba de seguridad → sombrero blanco / hacking ético
- Prueba de accesibilidad → ambientes de trabajo

Todas a nivel de sistema y aceptación
No todas a nivel de pruebas unitarias.

Caso de prueba: Situación descrita / detallada de una determinada funcionalidad que se quiere validar / verificar.

- Escenario [paso a paso]
- Datos de prueba → Precondiciones o condiciones de setup. Usar datos reales.
- Resultado esperado

Usar datos concretos en los casos de prueba.

- ¿Cuándo dejar de probar? → Buscar fortalecer la calidad de producto que estoy entregando
- El 100% nunca
 - Adivinación de defectos
 - Por tiempo dado se prueba lo que se llega (mala)
 - Métricas de porcentaje de errores.
 - Cuando corrí los casos de prueba planificados

Los ~~errores~~ ^{defectos} concuados van dentro de la nota de release.

Hitos

"Testing asegura la calidad" "Asegura que no hay defecto" "Le pone calidad al producto"

NO

La calidad es una decisión, no se agrega al final. Testing solo evidencia si hay o no.

NO se hace aseguramiento de calidad. Se hace control de calidad.

esto es durante todo el PUD.